CORRECTION — Mock DS — Python for Data Science

/20

Part 1: **theory, on paper** · Chapters 0, 1, 2 · Duration: 1 h · No documents, no computer

Part I — Multiple choice (6 pts · 0.5 per question · one correct answer)

Q1. Which phase of the software lifecycle checks that the program works correctly?

- ☐ a) Design
- ☑ **b) Testing**
- ☐ c) Deployment
- ☐ d) Maintenance

> Testing (Requirements → Design → Implementation → Testing → Deployment → Maintenance).

Q2. Which one is NOT one of the 5 components of an Information System?

- ☐ a) People
- ☐ b) Process
- ☑ **c) Internet**
- ☐ d) Data

> The 5 components are Hardware, Software, Data, People, Process.

Q3. The user types `20` in `age = input("Age: ")`. What is `type(age)`?

- ☐ a) int
- ☑ **b) str**
- ☐ c) float
- ☐ d) it depends on what is typed

> input() always returns a str.

Q4. `17 // 5` and `17 % 5` give:

- ☐ a) 3.4 and 2
- ☑ **b) 3 and 2**
- ☐ c) 3 and 3.4
- ☐ d) 2 and 3

> // = integer division (3), % = remainder (2).

Q5. Which object cannot be modified after its creation?

- ☐ a) list
- ☐ b) dict
- ☑ **c) tuple**
- ☐ d) NumPy array

> A tuple is immutable.

Q6. `np.array([2, 4.0, True]).dtype` is:

- ☐ a) int64
- ☐ b) bool
- ☑ **c) float64**
- ☐ d) <U32

> Type coercion: bool → int → float; the most general type present is float.

Q7. `np.arange(12).reshape((3, 5))`:

- ☐ a) returns a 3×5 array
- ☐ b) returns a 5×3 array
- ☑ **c) raises a ValueError**
- ☐ d) pads the array with zeros

> 12 elements cannot fill 3×5 = 15 places → ValueError.

Q8. Which pandas command shows the dtypes AND the number of non-null values per column?

- ☐ a) df.describe()
- ☑ **b) df.info()**
- ☐ c) df.head()
- ☐ d) df.shape

> info().

Q9. A DataFrame has 12,060 rows and 10 columns. `df.shape` is:

- ☐ a) 120600
- ☐ b) (10, 12060)
- ☑ **c) (12060, 10)**
- ☐ d) [12060, 10]

> A tuple (rows, columns); 120600 is df.size.

Q10. Which line correctly keeps the Tunis orders with more than 3 units?

- ☐ a) df[df.city == "Tunis" and df.q > 3]
- ☑ **b) df[(df["city"] == "Tunis") & (df["q"] > 3)]**
- ☐ c) df[df["city"] == "Tunis" & df["q"] > 3]
- ☐ d) df.loc["city" == "Tunis"]

> & between masks + parentheses around each condition.

Q11. `pd.to_datetime(s, errors="coerce")` on the value `"2025-13-40"`:

- ☐ a) raises an error
- ☑ **b) returns NaT**
- ☐ c) returns 2025-12-31
- ☐ d) keeps the string

> coerce → invalid date becomes NaT (missing).

Q12. With the default index (0, 1, 2…), how many rows does `df.iloc[0:3]` return? And `df.loc[0:3]`?

- ☐ a) 3 and 3
- ☑ **b) 3 and 4**
- ☐ c) 4 and 3
- ☐ d) 2 and 3

> iloc excludes the end (0,1,2); loc includes it (0,1,2,3).

Part II — What does this code print? (6 pts · 1 per snippet)

Write the exact output. If the code raises an error, give the name of the error.

P1.

```
x = "4"
y = 2
print(x * y, int(x) * y, int(x) / y)
```

```
44 8 2.0
```

> "4" * 2 repeats the string; / always gives a float.

P2.

```
total = 0
for i in range(2, 10, 3):
    total += i
    print(i, end=" ")
print("|", total)
```

```
2 5 8 | 15
```

> range(2, 10, 3) → 2, 5, 8 (10 excluded); end=" " keeps them on one line.

P3.

```
lst = [10, 20, 30, 40, 50]
lst.append(60)
print(lst[1:4], lst[-2], len(lst))
print(lst[::2])
```

```
[20, 30, 40] 50 6
[10, 30, 50]
```

slice end excluded; -2 = second from the end after the append.

P4.

```
d = {"Tunis": 3, "Sfax": 1}
d["Sousse"] = 2
d["Tunis"] += 1
for k, v in d.items():
    if v > 1:
        print(k, v)
```

```
Tunis 4
Sousse 2
```

a new key is added at the end; Sfax (1) is filtered out.

P5.

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]])
print(a.shape, a.size, a.sum(axis=0), a[1, 2])
print(a[a > 2])
```

```
(2, 3) 6 [5 7 9] 6
[3 4 5 6]
```

axis=0 = sum of each column; a[1, 2] = row 1, col 2; a mask returns a 1D array.

P6.

```
import pandas as pd
df = pd.DataFrame({"city": ["Tunis", " tunis", "Sfax", "Sfax"],
                   "q": [2, None, 5, 5]})
print(df.shape, df["q"].isna().sum(), df.duplicated().sum())
df["city"] = df["city"].str.strip().str.title()
print(df["city"].nunique(), df["q"].dtype, df["q"].mean())
```

```
(4, 2) 1 1
2 float64 4.0
```

None → NaN, so q is float64; the two 'Sfax, 5' rows are duplicates; mean ignores NaN: (2+5+5)/3 = 4.0.

Part III — Short questions (4 pts · 1 per question)

S1. The column `quantity` of a CSV contains whole numbers, but pandas loaded it as `float64` (1.0, 2.0…). Why? What must you do before `astype("int64")`?

> The column contains **missing values**: NaN is a float, so the whole column becomes float64. Before converting, handle the NaN (`dropna(subset=["quantity"])` or `fillna(...)`), otherwise `astype("int64")` raises an error.

S2. Give two different treatments for missing values with an example of when each is appropriate. Why must we not replace all missing values by 0?

> • **Fill**: a category → a label (`payment_method → "Unknown"`) or the mode; a number with a business rule → median of the group (missing price → median price of the same product). • **Drop the rows**: when a key field is missing (`order_date`), with `dropna(subset=[...])`. (Also: drop the column if most values are missing, after asking the business.) **Not 0**: it creates fake values (price 0, age 0) and biases the averages. Every choice must be documented.

S3. Explain the difference between `df.loc` and `df.iloc`. Give an example where they return different rows.

> `loc` selects by **labels** (index labels, column names, masks) and its slices **include** the end; `iloc` selects by **integer positions** and excludes the end. After a filter the index has gaps: if the labels are 0, 2, 5, 7, then `df.loc[2:5]` returns labels 2 and 5, while `df.iloc[2:5]` returns positions 2–3 → labels 5 and 7. Even with 0…n: `loc[0:3]` = 4 rows, `iloc[0:3]` = 3 rows.

S4. What does `orders.groupby("product_id")["unit_price"].transform("median")` return, compared with `.median()`? Why is it useful for cleaning?

> `.median()` returns **one value per product** (a short Series indexed by product_id). `.transform("median")` returns a Series of the **same length as orders**, aligned with its rows: each row gets the median of its own product. It can be passed directly to `fillna`: `orders["unit_price"].fillna(orders.groupby("product_id")["unit_price"].transform("median"))`.

Part IV — Write the code by hand (4 pts)

C1. Write a function `grade(score)` that returns `"Excellent"` if score ≥ 16, `"Good"` if ≥ 12, `"Pass"` if ≥ 10 and `"Fail"` otherwise. Then, for `scores = [8.5, 12, 17.25, 10, 9.75, 14]`, print each score with its grade and print how many students passed (score ≥ 10) and the class average. 2 pts

```
def grade(score):
    if score >= 16:
        return "Excellent"
    elif score >= 12:
        return "Good"
    elif score >= 10:
        return "Pass"
    else:
        return "Fail"

scores = [8.5, 12, 17.25, 10, 9.75, 14]
passed = 0
for s in scores:
    print(s, "->", grade(s))
    if s >= 10:
        passed += 1
print("Passed:", passed)                          # 4
print("Average:", sum(scores) / len(scores))      # 11.916666...
```

C2. A DataFrame `orders` has the columns `order_id, product_id, city, quantity, unit_price`. Write the code to: (a) remove the duplicated rows; (b) clean `city` (spaces and upper/lower case); (c) keep only the lines with at most 10 units; (d) fill the missing `unit_price` with the median price of the same product; (e) create `revenue` = quantity × unit_price; (f) display the total revenue per city, from highest to lowest; (g) export the result to `orders_clean.csv` without the index. 2 pts

```
orders = orders.drop_duplicates()                                    # (a)
orders["city"] = orders["city"].str.strip().str.title()              # (b)
orders = orders[orders["quantity"] <= 10].copy()                     # (c)
med = orders.groupby("product_id")["unit_price"].transform("median")
orders["unit_price"] = orders["unit_price"].fillna(med)              # (d)
orders["revenue"] = orders["quantity"] * orders["unit_price"]        # (e)
print(orders.groupby("city")["revenue"].sum().sort_values(ascending=False))   # (f)
orders.to_csv("orders_clean.csv", index=False)                       # (g)
```

Marking scheme

Part	Points	How it is marked
I	6	0.5 per correct answer, 0 if wrong or several boxes
II	6	1 per snippet: exact output (0.5 if one value is wrong)
III	4	idea (0.5) + example / justification (0.5)
IV	4	C1: function + if/elif (1), loop + counter + average (1). C2: 0.25–0.5 per step, syntax errors −0.25